CCTV 차량 탐지 기반

정체 판단 서비스 개발 과정

기획 → 데이터 구축 → 모델 선정 → 모델 학습 → 성능 개선 → API 구현

기획 — 교통 정체라는 사회적 문제

PROBLEM 교통량 증가가 정체·사고로 이어지고 있음

"2023년 교통혼잡비용 81조 3천억원, 전년 대비 9.0%↑"

한국교통연구원·국토교통부 (GDP의 3.37%)

"국토부 "2025년 일교통량 0.9%↑…자동차 등록 증가 영향""

서울경제 (2026.5.8)

"올해 1~5월 고속도로 교통사고 사망자 83명, 전년보다 70%↑"

부산일보

SOLUTION 교통 흐름 분석으로 정체를 줄인 실제 사례

"제주, AI·디지털트윈 기술로 도령로·노형로 교통혼잡 해소하며 77억원 절감"

보안뉴스 (2025.12.2)

"AI 신호제어 'AITOBAHN' 국토부 실증 성공…교통 문제 해결 '청신호'"

삼다일보 (2025.5.18)

해결 사례	핵심 지표	결과
제주 디지털트윈 (도령로·노형로)	통행시간	9.4%↓ (연 77억원 절감)
AITOBAHN (국토부 실증)	교통 흐름	17~24% 개선

▼ 교통 흐름을 분석하면 정체를 줄일 수 있다는 게 실증됐다 — **이 문제를 CCTV 차량 탐지 기반 정체 판단 API로 풀고자 기획함**

1차 데이터·모델 조사

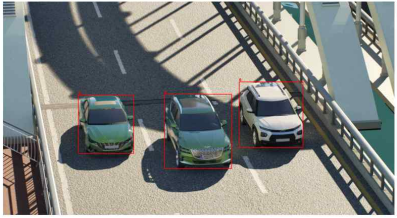
2차 학습·성능 개선

3차 API 서비스화

4차 종합 보고(본 발표)

데이터 구축 — 데이터 현황과 검증

항목	수치
전체 이미지 수	1,995장 (데이콘 synthetic)
train / val / test	1,197 / 399 / 399 (6:2:2)
클래스 수	1개 (car 단일 클래스)
원본 해상도	1920 × 1040
이미지당 평균 객체 수	2.63대 (최소1~최대3)
bbox 크기(정규화)	width 14.6% · height 28.5%
차량 간 겹침(IoU>0)	5.6% (대부분 비겹침)
라벨 검증(이미지↔라벨 차집합)	누락 0건
bbox 중심 좌표 분포	x축 3개 군집(3차선) 발견



실제 라벨 검증 화면 — 변환된 bbox를 이미지 위에 그려 육안 확인

모델 선정 — 왜 YOLO26인가

다중 CCTV 실시간 처리 요구사항을 기준으로, 대표 2-stage 모델(Faster R-CNN)·최신 실시간 대안(RT-DETR)과 비교 검증

항목	YOLO26	Faster R-CNN	RT-DETR
구조	1-stage (동시 예측)	2-stage (제안→분류)	Transformer 기반
mAP (COCO)	40.9 ~ 57.5 (x스케일)	37.0	53.1 ~ 54.3
속도 (GPU)	85 ~ 588 FPS	약 26 FPS (V100)	74 ~ 108 FPS
유지보수	활발 (릴리즈 438회)	-	연구 공개 코드 수준
결론	최종 채택	배제 — 2-stage 구조상 실시간 기준 미충족	차선 — 속도·유지보수 모두 YOLO26 대비 열위

** x스케일 = YOLO 모델 크기(n<s<m<l<x) 중 가장 큰 버전 — 작을수록 빠르고 덜 정확, 클수록 느리고 더 정확*

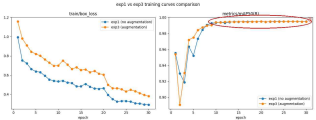
최종 선정: YOLO26 — 세 기준 모두 RT-DETR 대비 우위

- 실시간성 — 최대 588FPS, RT-DETR(74~108FPS) 대비 압도적으로 빠름
- 성능 — mAP 57.5(x스케일)로 RT-DETR(54.3) 대비 근소 우위
- 개발 효율성 — 릴리즈 438회로 RT-DETR(연구 공개 코드 수준) 대비 유지보수 압도적

모델 학습 & 성능 개선 — 실제 실행 내역

epochs=10 학습 후 GT vs Pred를 직접 대조해 오류를 발견 → 원인을 하나씩 검증하며 값을 조정한 개선 과정

단계	실행 내용	결과·원인
① **Baseline**	epochs=10 학습 후 평가	TP99·FP7·FN4 (P0.934, R0.961) → 오류 원인 파악 필요
② **원인분석**	GT/Pred 좌표 직접 대조	FP 7건 중 6건 중복탐지 (IoU≈0.99, NMS 무반응) FN은 후보 자체가 없음
③ **실험1** **(FN 해결 시도)**	epochs 10→30	TP103·FP1·FN0 FN 완전 해결 (Recall 1.000)
④ **실험2** **(FP 해결 시도)**	conf 0.25→0.45	남은 FP도 0건 P1.000·R1.000 달성
⑤ **실전테스트**	실제 사진 6장 추론	0~2대만 탐지 → 각도 domain gap 확인
⑥ **실험3** **(일반화 개선)**	augmentation 추가 재학습	실사진 3장 개선 synthetic 소폭 하락



실험1 vs 실험3 학습 곡선 비교 — ① epoch10~ 두 실험 모두 거의 동일하게 수렴



이전 — baseline, FP(중복탐지) 발생



이후 — 실험3(최종모델), 정상 매칭

leakage 검증: 전체 1,995장 MD5 해시 비교 → 중복 0건 · 최종 모델: 실험3(실전 일반화 우선)

API 구현 — 설정 근거와 실제 테스트

POST /detect — 최종 모델(conf=0.45, imgsz=224)

- vehicle_cnt · area_ratio(면적비율%) · avg_gap_px(평균간격) · congestion_level 반환

지표는 실제 물리량이 아닌 Proxy 지표

- area_ratio: 박스 겹침·대형차 포함 시 과대평가 가능 → '상대적 정체 경향'
- avg_gap_px: 카메라 원근에 좌우됨 → '상대적 간격 경향'

Threshold는 어떻게 잡았나 (감 → 데이터 검증)

- AI-Hub CCTV 750장 GT(정답 라벨) 기준 분포: 중앙값 1.97% · 90분위 5.78%
- 기존 5%/15% 기준: congested 0.1%뿐(죽은 등급) → 경계 구간 육안 확인 후 2%/8.5%로 재설정

기준	smooth	moderate	congested
기존 (5%/15%)	85.9%	14.0%	0.1%
신규 (2%/8.5%)	50.8%	46.3%	2.9%

실제 배포된 API(ngrok)에 curl로 직접 요청 — 최종 모델(exp3)

- → AI-Hub CCTV 원본을 넣자 0대 탐지(완전 실패) → 각도가 다른 사진으로
- 바꿔 재요청하니 정상 동작 — 배포 환경에서도 각도 domain gap 재확인

실패 사례 — 실제 API 응답





AI-Hub CCTV 원본(삼정고가교) — 실제론 20대+ 정체인데 vehicle_count=0, congestion=smooth로 완전히 놓침

성공 사례 — 실제 API 응답





각도가 다른(수직에 가까운) 사진 — vehicle_count=6, congested로 등급까지 정상 판정

06 CONCLUSION

결론 — 아쉬운 점 & 배운 점

아쉬운 점 (한계)

- 실제 사진 검증 표본이 6장뿐 → 통계적 확정 어려움
- augmentation은 각도 domain gap을 완화만, 완전 해소는 아님
- 학습 데이터에 실제 CCTV 각도 이미지가 전무 (synthetic 항공뷰뿐)
- threshold(2%/8.5%)는 CCTV 1개 지점 기준 — 타지점 미검증
- threshold는 GT 기준, API는 모델 예측 기준 — 모델 오탐/미탐 시 둘이 어긋날 수 있음(미검증)

배운 점 (다음엔 이렇게)

- '빠른 실험' ≠ '데이터 축소' — fraction 실패로 체감
- 라이브러리 옵션이 항상 기대대로 동작하지 않음 (NMS 무반응)
- synthetic 성능을 곧이곧대로 믿지 않고 실전 재검증하는 습관
- threshold 등 숫자는 데이터 분포로 검증해야 근거가 생김

다음 프로젝트에서 우선 시도할 것

- ① 실제 각도 CCTV 이미지 소량 라벨링·fine-tuning ② 타 지점 threshold 재검증 ③ 역광·대형차 데이터 보강